SIMATS ENGINEERING

CO2_AT2 Reverse Engineering Task

COURSE NAME: Software Testing **COURSE CODE: CSA37**

COURSE OUTCOME COVERED

CO2: *Design effective test cases using black-box, white-box testing techniques and to control methods flow.*

Sl. No	Questions	Bloom's Level
1	A partially tested ATM withdrawal module is provided where only input-output behavior is known (e.g., withdrawal limits, balance validation, error messages). Reverse engineer the system by analyzing the observed behavior and reconstructing the underlying test case design strategy. Decompose the system into functional components and identify possible equivalence classes and boundary conditions. Infer the black box testing techniques used and evaluate their adequacy. Reconstruct an improved set of test cases covering all critical scenarios. Document your analysis with structured explanation.	5
2	A software module's execution traces and code snippets are given, but no test documentation is available. Reverse engineer the white box testing approach by constructing the control flow graph from the given logic. Identify all decision points, loops, and independent execution paths. Evaluate the existing test adequacy in terms of coverage criteria. Reconstruct a complete path testing strategy ensuring full coverage. Present your findings with clear reasoning and structured representation.	5
3	A web application login and transaction system has been tested using unknown techniques, but only defect logs and test results are available. Reverse engineer the testing approach by analyzing defect patterns and identifying which black box and white box techniques were likely used. Decompose the system into testable components and evaluate gaps in testing coverage. Reconstruct a comprehensive testing strategy integrating equivalence partitioning, boundary value analysis, and path testing. Justify your reconstructed approach with logical reasoning and structured documentation.	5

4	A legacy inventory management system provides only a set of successful and failed transaction logs regarding stock updates and low-stock alerts. Reverse engineer the business logic by analyzing these logs to identify the hidden boundary conditions and equivalence partitions. Decompose the update process into functional units and reconstruct a black box test design that addresses the gaps found in the logs. Justify your selection of test cases based on the inferred logic.	5
5	You are provided with a compiled .exe file of a library management module and its corresponding CPU/Memory execution profile during a stress test. Reverse engineer the white box testing requirements by mapping execution spikes to potential loops and decision clusters in the logic. Construct a conceptual control flow graph (CFG) that represents this behavior. Reconstruct a path testing strategy that targets these resource-intensive paths and evaluate its impact on system stability.	6
6	A mobile payment gateway's API responses (JSON) for various transaction states—Pending, Success, Reversal, and Timeout—are given. Reverse engineer the underlying state machine by analyzing the transition triggers and data constraints. Identify the implicit state transition testing technique used. Reconstruct a comprehensive state-based test suite that ensures all legal and illegal transitions are covered. Document your analysis with a structured state-transition diagram.	5
7	An encrypted communication module provides only "black-box" behavior through its input parameters and encrypted outputs. Reverse engineer the validation logic by performing "fuzzing" analysis on the input fields to identify error-handling boundaries. Decompose the module into data validation and encryption components. Reconstruct an improved set of test cases using Boundary Value Analysis (BVA) and justify how your design prevents buffer overflow or injection vulnerabilities.	5
8	A set of legacy unit test results for an insurance premium calculator shows 100% statement coverage but fails to catch defects in multi-condition "if" statements. Reverse engineer the original white box approach and identify the inadequacy in the coverage criteria used. Construct a new CFG and identify all independent execution paths. Reconstruct a path testing strategy that achieves 100% Branch and Path coverage. Explain how this reconstruction reduces the risk of calculation errors.	5

9	A cloud-based file synchronization tool provides user activity logs and a list of "lost file" defect reports. Reverse engineer the synchronization algorithm's logic by analyzing the defect patterns related to file size, type, and network speed. Infer the likely Equivalence Class Partitioning (ECP) used by the original developers. Reconstruct a revised testing strategy that integrates BVA with stress testing. Justify how your reconstructed approach prevents future data loss.	5
10	You are given a sequence of database queries (SQL) generated by an e-commerce "Search and Filter" module, but no source code. Reverse engineer the search logic by analyzing the query parameters and the resulting dataset. Identify the decision-making logic used for complex filtering (e.g., AND/OR/NOT). Reconstruct a Decision Table based testing strategy that covers all possible search combinations. Evaluate the completeness of your reconstructed model.	5

Assessment Tool Weightage: 30%

Code of Conduct:

I **RenugaDevi B (192311111)** *(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:
RenugaDevi B

RUBRICS FOR CALCULATION:3

Criteria	Weightage (%)	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Understanding of System	20%	Demonstrates clear and in-depth understanding of the system/product and its functionality	Shows good understanding with minor gaps	Partial understanding; misses key aspects	Lacks understanding of the system/product
Decomposition	25%	Effectively breaks down the system into components with detailed analysis	Decomposes system with minor gaps in analysis	Limited or incomplete decomposition	Unable to analyze or break down system
Identification of Components	20%	Accurately identifies all components and explains their functions clearly	Identifies most components with minor errors	Identifies few components; explanations are weak	Unable to identify components or functions
Reconstruction	20%	Successfully reconstructs or models the system with accuracy and logic	Reconstruction is mostly correct with minor issues	Partial reconstruction; lacks clarity	Unable to reconstruct or model system
Documentation	15%	Well-organized, clear documentation with diagrams and structured explanation	Documentation is adequate with minor issues	Poorly organized or incomplete documentation	No proper documentation or presentation

1. ATM Withdrawal Module – Black Box Testing and Reverse Engineering

Introduction

Reverse engineering is the process of understanding a software system by analyzing its inputs, outputs, and observed behavior without knowing its internal code. In an ATM withdrawal module, withdrawal limits, balance validation, and error messages help us reconstruct the original testing strategy.

1. Functional Decomposition

The ATM withdrawal module consists of:

1. **PIN Validation:** Checks whether the entered PIN is correct.
2. **Account Validation:** Verifies whether the account is active.
3. **Balance Checking:** Checks whether sufficient funds are available.
4. **Withdrawal Limit Checking:** Validates transaction and daily limits.
5. **Cash Dispensing:** Dispenses money after successful validation.
6. **Error Handling:** Displays appropriate error messages.

2. Reverse Engineering and Testing Techniques

The observed behavior helps identify the likely testing approach. If successful and unsuccessful withdrawals have been tested, basic equivalence partitioning may have been used. If values near withdrawal limits were tested, boundary value analysis may have been applied.

Equivalence Partitioning (ECP): Divides inputs into valid and invalid classes.

Boundary Value Analysis (BVA): Tests values around important limits.

For example, if the maximum withdrawal limit is ₹20,000, test ₹19,999, ₹20,000, and ₹20,001.

3. Reconstructed Test Cases

Test	Input	Expected Result
1	Valid PIN and sufficient balance	Withdrawal successful
2	Incorrect PIN	Error message
3	Zero withdrawal amount	Invalid amount
4	Amount greater than balance	Transaction rejected
5	Amount equal to daily limit	Accepted if permitted
6	Amount exceeding daily limit	Transaction rejected
7	Negative withdrawal amount	Invalid amount

4. Evaluation and Improvement

Testing only successful transactions is insufficient. The improved approach should include boundary values, invalid inputs, account validation, and error handling. It should also verify that failed transactions do not deduct money.

Conclusion

Reverse engineering helps reconstruct the original test design from observable behavior. Combining equivalence partitioning and boundary value analysis improves coverage and ensures reliable ATM operation.

2. White Box Testing – Control Flow Graph and Path Testing

Introduction

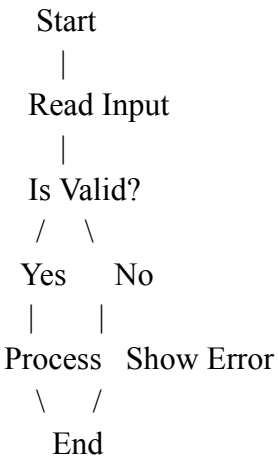
White box testing examines the internal program logic. When test documentation is unavailable, execution traces and code snippets can help reconstruct the original testing strategy.

1. Constructing the Control Flow Graph

A Control Flow Graph (CFG) represents the flow of execution in a program.

- **Nodes:** Statements or groups of statements.
- **Edges:** Connections between statements.
- **Decision nodes:** Conditions such as `if` and `else`.
- **Loops:** Statements that execute repeatedly.

Consider this example:



The graph contains two possible execution paths.

2. Identifying Independent Paths

- **Path 1:** Start → Read Input → Valid → Process → End.
- **Path 2:** Start → Read Input → Invalid → Show Error → End.

Each path must be tested using suitable inputs.

3. Cyclomatic Complexity

Cyclomatic complexity estimates the number of independent paths.

$$V(G) = E - N + 2P$$

Where:

- E = Number of edges.
- N = Number of nodes.
- P = Number of connected components.

For a single connected graph:

$$V(G) = E - N + 2$$

For the example above, there are 7 edges and 6 nodes.

$$V(G) = 7 - 6 + 2 = 3$$

4. Coverage Evaluation

- **Statement Coverage:** Executes every statement.
- **Branch Coverage:** Tests true and false outcomes of decisions.
- **Path Testing:** Tests independent execution paths.
- **Condition Coverage:** Tests individual Boolean conditions.

5. Improved Testing Strategy

1. Identify statements, decisions, and loops.
2. Draw the CFG.
3. Calculate cyclomatic complexity.
4. Design test cases for independent paths.
5. Execute the tests and identify uncovered branches.

Conclusion

CFG-based path testing helps identify missing tests and improves internal logic coverage. Complete path coverage may not be possible when programs contain unrestricted loops.

3. Login and Transaction System – Combined Testing

Introduction

A web application containing login and transaction features must be tested for correctness, security, and reliability. Defect logs and test results help identify the techniques previously used and the gaps in testing.

1. Functional Decomposition

The system contains:

1. Login and authentication.
2. Session management.
3. Transaction processing.
4. Balance validation.
5. Error handling.

2. Reverse Engineering the Testing Approach

Defect patterns provide clues about the original approach.

- Incorrect login results may indicate insufficient authentication testing.
- Incorrect balance calculations may indicate missing transaction validation.
- Defects near transaction limits may indicate inadequate boundary value analysis.
- Errors in particular execution paths may indicate insufficient branch testing.

These are possible explanations; defect logs alone cannot prove which technique was originally used.

Black Box Testing: Equivalence partitioning separates valid and invalid credentials. Boundary value analysis tests transaction limits.

White Box Testing: Branch and path testing examine internal decision outcomes and execution paths.

3. Evaluation and Improvement

The improved approach combines equivalence partitioning, boundary value analysis, branch coverage, and path testing. Security tests should also verify unauthorized access and session expiration.

4. Reconstructed Test Cases

Test	Input/Condition	Expected Result
1	Correct credentials	Login successful
2	Incorrect password	Login rejected
3	Empty login fields	Validation message
4	Valid transaction within limit	Transaction successful
5	Transaction exceeds balance	Transaction rejected
6	Transaction at maximum limit	Correct limit handling
7	Expired session	Login required
8	Repeated transaction request	Duplicate processing prevented where required

Conclusion

Combining black box and white box testing improves functional and code coverage. It helps detect authentication defects, incorrect calculations, and transaction failures.

4. Inventory Management System – Equivalence Partitioning and Boundary Testing

Introduction

An inventory system maintains stock quantities and generates low-stock alerts. Successful and failed transaction logs can help identify hidden business rules and reconstruct the original test strategy.

1. Functional Decomposition

The stock update process includes:

1. Product validation.
2. Quantity validation.
3. Stock updating.
4. Low-stock alert generation.

2. Reverse Engineering from Transaction Logs

If logs show that stock updates succeed for positive quantities but fail for negative quantities, the system probably validates the stock quantity. If alerts appear for some quantities but not others, the alert threshold can be inferred by comparing the results. However, the exact threshold and whether it uses $<$ or $\leq$ must be confirmed through additional tests.

3. Equivalence Partitioning

Possible input classes include:

- Valid positive quantities.
- Zero quantities.
- Negative quantities.
- Quantities exceeding the permitted limit.
- Existing product IDs.
- Invalid product IDs.

4. Boundary Value Analysis

Assume the low-stock threshold is 10 units.

Stock Quantity	Expected Result
9	Alert generated
10	Alert generated if the rule is quantity ≤ 10
11	No alert
0	Out-of-stock condition handled
−1	Invalid quantity rejected

5. Reconstructed Test Cases

1. Add stock to an existing product.
2. Reduce stock without making it negative.
3. Attempt to reduce stock below zero.
4. Update a nonexistent product.
5. Set stock to the alert threshold.
6. Reduce stock below the threshold.
7. Verify that failed updates do not change the stored quantity.

6. Evaluation and Improvement

The reconstructed tests combine equivalence partitioning, boundary value analysis, and database consistency checks. They address gaps that ordinary successful transactions may not reveal.

Conclusion

Reverse engineering transaction logs helps identify hidden stock rules. The improved test suite reduces incorrect stock updates and ensures reliable low-stock alerts.

5. Library Management EXE – Resource Analysis and Path Testing

Introduction

A compiled EXE file does not directly reveal its source code. When CPU and memory profiles are available, reverse engineering can use execution behavior to identify possible performance problems and develop a conceptual testing strategy. CPU and memory spikes provide clues, but they cannot confirm the exact internal program logic.

1. Functional Decomposition

The library management module may contain:

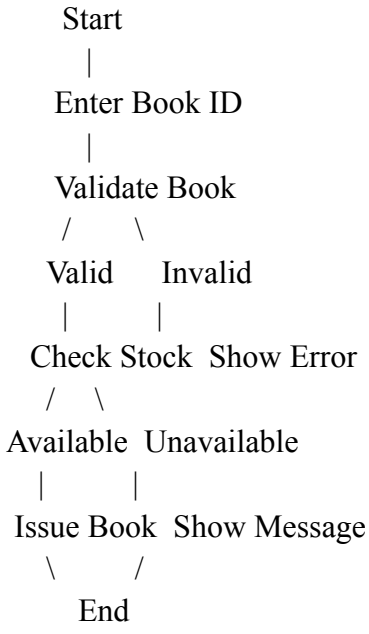
1. Book searching.
2. Book issuing.
3. Book returns.
4. Member validation.
5. Fine calculation.
6. Database updates.

2. Analyzing Execution Profiles

- **High CPU usage:** May indicate repeated calculations, inefficient searches, or loops.
- **High memory usage:** May indicate large data structures, repeated allocations, or memory leaks.
- **Increasing memory over repeated operations:** May suggest that memory is not being released correctly.

For example, searching thousands of book records may require more processing than searching a small collection.

3. Conceptual Control Flow Graph



This is a possible model based on the module's expected behavior, not a confirmed representation of its internal code.

4. Reconstructed Path Testing Strategy

1. Test a valid book ID with an available book.
2. Test a valid book ID with an unavailable book.
3. Test an invalid book ID.
4. Test an invalid member attempting to issue a book.
5. Repeat issue and return operations.
6. Search a large collection of records.
7. Monitor CPU, memory, and response time under stress.

5. Evaluation and Improvement

The tests can reveal slow operations, crashes, and possible resource leaks. Binary analysis, debugging tools, or additional execution evidence may be needed to reconstruct the actual CFG.

Conclusion

Combining conceptual path testing with stress testing helps identify performance bottlenecks and improves system stability. Resource profiles alone are insufficient to prove the exact control flow.

6. Mobile Payment Gateway – State Transition Testing

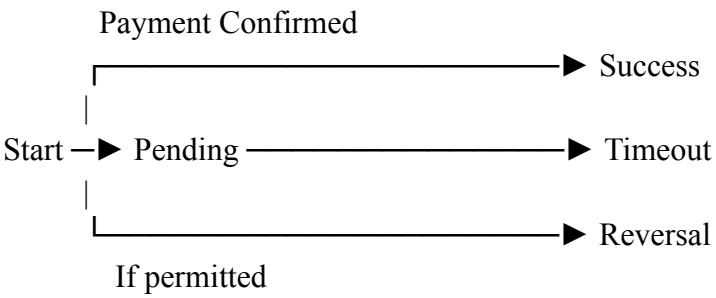
Introduction

A mobile payment gateway processes transactions through different states. State Transition Testing checks whether the system moves correctly between states when specific events occur.The main states are Pending, Success, Reversal, and Timeout.

1. Identifying the States

- **Pending:** The transaction has started but is incomplete.
- **Success:** The payment has completed successfully.
- **Reversal:** A previous transaction is reversed according to the payment rules.
- **Timeout:** The expected response was not received within the permitted time.

2. State Transition Diagram



3. Reconstructed State Transition Test Cases

Current State	Event	Expected Next State
Pending	Payment confirmed	Success
Pending	Response timeout	Timeout
Pending	Valid cancellation	Reversal or cancellation state, as specified
Success	Valid reversal request	Reversal, if permitted
Timeout	Late payment confirmation	Reconciled according to defined rules
Any state	Invalid transition request	Rejected or safely handled

4. Testing Data Constraints

Each request should contain a valid transaction ID, amount, and required fields. Invalid requests must be rejected, and duplicate requests must not cause duplicate payments. For example, when a transaction times out but the bank later confirms payment, the gateway should reconcile the transaction rather than charge the customer twice.

5. Evaluation and Improvement

The improved suite should cover all legal transitions, important illegal transitions, repeated requests, and timeout handling. API responses must match the expected transaction states.

Conclusion

State Transition Testing detects incorrect transaction states, duplicate payments, and reversal errors. It improves the reliability of mobile payment processing.

7. Encrypted Communication Module – Fuzzing and Boundary Value Analysis

Introduction

An encrypted communication module accepts input data and produces encrypted output. When source code is unavailable, black box testing and fuzzing can identify input-validation errors and unexpected behavior.

1. Functional Decomposition

The module contains:

1. Input validation.
2. Data processing.
3. Encryption.
4. Error handling.

2. Reverse Engineering the Validation Logic

Fuzzing results can reveal how the module responds to unexpected inputs.

For example:

- If short inputs are accepted but empty inputs are rejected, a minimum input length may exist.
- If very long inputs produce errors, a maximum length may be enforced.
- If malformed inputs cause crashes, the validation logic may be inadequate.

These observations suggest possible validation boundaries, which must be confirmed through repeated tests.

3. Boundary Value Analysis

Assume the permitted input length is between 1 and 100 characters.

Test the following values:

- 0 characters.
- 1 character.
- 2 characters.
- 99 characters.
- 100 characters.
- 101 characters.

The system should accept valid lengths and safely reject inputs outside the specified range.

4. Reconstructed Test Cases

Test	Input	Expected Result
1	Valid input	Encrypted output generated
2	Empty input	Rejected if prohibited
3	Maximum permitted length	Processed successfully
4	Input exceeding maximum length	Safely rejected
5	Malformed input	Validation error
6	Very large repeated inputs	No crash or uncontrolled resource usage
7	Incorrect encryption parameters	Safe error response

5. Evaluation and Improvement

Fuzzing and boundary testing can reveal possible memory-handling and malformed-input vulnerabilities. However, they cannot guarantee protection against buffer overflows or injection attacks. Additional security testing should examine memory safety, parameter validation, encryption implementation, and error handling.

Conclusion

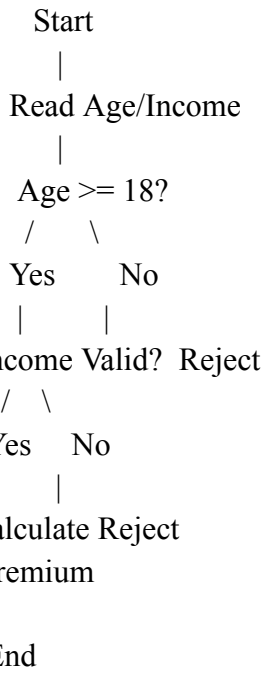
Combining fuzzing with boundary value analysis improves input validation and error handling. It helps identify security weaknesses before deployment.

8. Insurance Premium Calculator – Branch Coverage and Path Testing

Introduction

An insurance premium calculator calculates premiums based on factors such as age, income, and policy type. If existing tests achieve 100% statement coverage but still miss defects, the testing strategy may be inadequate. Statement coverage ensures that each statement executes, but it does not guarantee that all decision outcomes or condition combinations have been tested.

1. Example Program Logic



This conceptual CFG shows different execution paths through the calculator.

2. Identifying Independent Paths

- **Path 1:** Eligible age → Valid income → Calculate premium.
- **Path 2:** Eligible age → Invalid income → Reject input.
- **Path 3:** Ineligible age → Reject applicant.

Each path should be tested with appropriate input values.

3. Coverage Evaluation

- **Statement Coverage:** Executes each statement.
- **Branch Coverage:** Tests true and false outcomes of decisions.
- **Condition Coverage:** Tests each Boolean condition as true and false.
- **Path Testing:** Tests independent execution paths.

For example, a program may execute the premium calculation statement without adequately testing every condition that determines the premium amount.

4. Reconstructed Test Cases

1. Enter eligible age and valid income.
2. Enter age below the minimum requirement.
3. Enter the minimum eligible age.
4. Enter valid age with invalid income.
5. Test different policy types.
6. Test combinations of age, income, and policy conditions.
7. Compare calculated premiums with expected values.

5. Evaluation and Improvement

The improved approach should measure branch and condition coverage and test independent paths using the CFG. For compound conditions, Modified Condition/Decision Coverage (MC/DC) may also be useful because it checks whether each condition can independently affect the overall decision outcome. Complete path coverage may be impractical when a program has many combinations or loops.

Conclusion

The improved white box testing strategy detects defects that statement coverage may miss. It helps ensure correct decisions and reduces the risk of incorrect premium calculations.

9. Cloud File Synchronization – Equivalence Partitioning and Stress Testing

Introduction

A cloud file synchronization tool transfers files between devices and cloud storage. Lost-file defects may occur because of file size, unsupported formats, network interruptions, or synchronization failures. Activity logs and defect reports help identify weaknesses in the existing test strategy.

1. Functional Decomposition

The system includes:

1. File selection and validation.
2. File size and type checking.
3. Network management.

4. Uploading and downloading.
5. Synchronization status tracking.
6. File integrity verification.

2. Reverse Engineering from Defect Reports

If lost-file reports occur mainly during network interruptions, the retry or recovery mechanism may be inadequate.

If failures occur with large files, the file-size validation or transfer process may require further testing.

If certain file types repeatedly fail, the original equivalence classes may not cover all supported and unsupported formats.

These are possible explanations that should be verified through controlled testing.

3. Equivalence Partitioning

Possible classes include:

- Small, medium, and large files.
- Supported and unsupported file formats.
- Stable, slow, and interrupted networks.
- Successful and failed synchronization attempts.

4. Boundary Value Analysis

Assume the maximum permitted file size is 100 MB.

File Size	Expected Result
99 MB	Accepted
100 MB	Accepted if the limit is inclusive
101 MB	Rejected

5. Reconstructed Test Cases

1. Synchronize a small file using a stable network.
2. Upload a file at the maximum permitted size.
3. Attempt to upload an oversized file.
4. Synchronize an unsupported file type.
5. Interrupt the network during transfer.
6. Resume an interrupted transfer.
7. Synchronize many files simultaneously.
8. Compare file checksums after synchronization.

6. Stress Testing and Improvement

Stress testing evaluates the system under heavy workloads. It helps identify slow transfers, crashes, and resource problems. Checksums and file-integrity comparisons help confirm that files are neither missing nor corrupted. Retry mechanisms should also avoid creating duplicate files.

Conclusion

Combining equivalence partitioning, boundary value analysis, and stress testing improves synchronization reliability. It helps identify file-loss defects and ensures correct behavior under different file and network conditions.

10. E-commerce Search and Filter – Decision Table Testing

Introduction

An e-commerce search and filter module helps users find products using keywords, categories, prices, and availability. SQL queries and their results can be analyzed to reconstruct the filtering logic. Decision Table Testing is useful for checking combinations of conditions and their expected results.

1. Functional Decomposition

The module contains:

1. Keyword searching.
2. Category selection.
3. Price filtering.
4. Product availability checking.
5. Sorting and displaying results.

2. Reverse Engineering the Filtering Logic

The SQL queries and returned products provide clues about the filtering conditions.

- **AND:** All selected conditions must be satisfied.
- **OR:** At least one specified condition must be satisfied.
- **NOT:** Products matching a specified condition are excluded.

By comparing queries and results, we can identify which conditions are applied and whether the filtering behavior matches the expected rules.

3. Example Decision Table

Assume the system uses AND logic for the selected keyword, category, and price filters.

Rule	Keyword Match	Category Match	Price Match	Expected Result
R1	Yes	Yes	Yes	Display product
R2	Yes	Yes	No	Exclude product
R3	Yes	No	Yes	Exclude product
R4	No	Yes	Yes	Exclude product
R5	No	No	No	No matching products

This table illustrates AND logic. If the application uses OR logic, the expected results must be adjusted accordingly.

4. Reconstructed Test Cases

1. Search using a valid keyword.
2. Filter products by category.
3. Apply minimum and maximum price limits.
4. Combine keyword and category filters.
5. Verify AND logic.
6. Verify OR logic.
7. Exclude products using NOT logic.
8. Enter conflicting filters.
9. Search without selecting any filters.
10. Test invalid price ranges and boundary values.

5. Evaluation and Improvement

The actual search results should be compared with the decision table. Missing combinations may reveal incorrect SQL conditions or improper AND/OR operators. When many conditions exist, the number of combinations becomes large. Rule reduction and pairwise testing can reduce the number of tests while maintaining useful coverage.

Conclusion

Decision Table Testing provides a systematic way to validate complex filtering conditions. Combining it with equivalence partitioning and boundary value analysis improves search accuracy and ensures that products are displayed according to the selected filters.